



NATIONAL INSTITUTE OF TECHNICAL TEACHERS' TRAINING & RESEARCH, KOLKATA

Deemed to be University

Department of Computer Science & Engineering

Program: M. Tech. in AI & ML with Pedagogy

DOCUMENTATION OF WEEKEND CHALLENGE

Course Name: Artificial Intelligence Lab

Course Code: AIMLPC107

Name: Anish Dey

Semester: 1st

Roll No.: 1330261012

Registration No.:

Session: 2026-27

Faculty's Signature

Student's Signature

1. PROBLEM STATEMENT

"Develop a web application of De Bruijn graph for Genome Construction" using attached data.

2. BRAINSTORM

The brainstorming process focuses on identifying the key steps required to construct and visualize a genome using the De Bruijn graph approach.

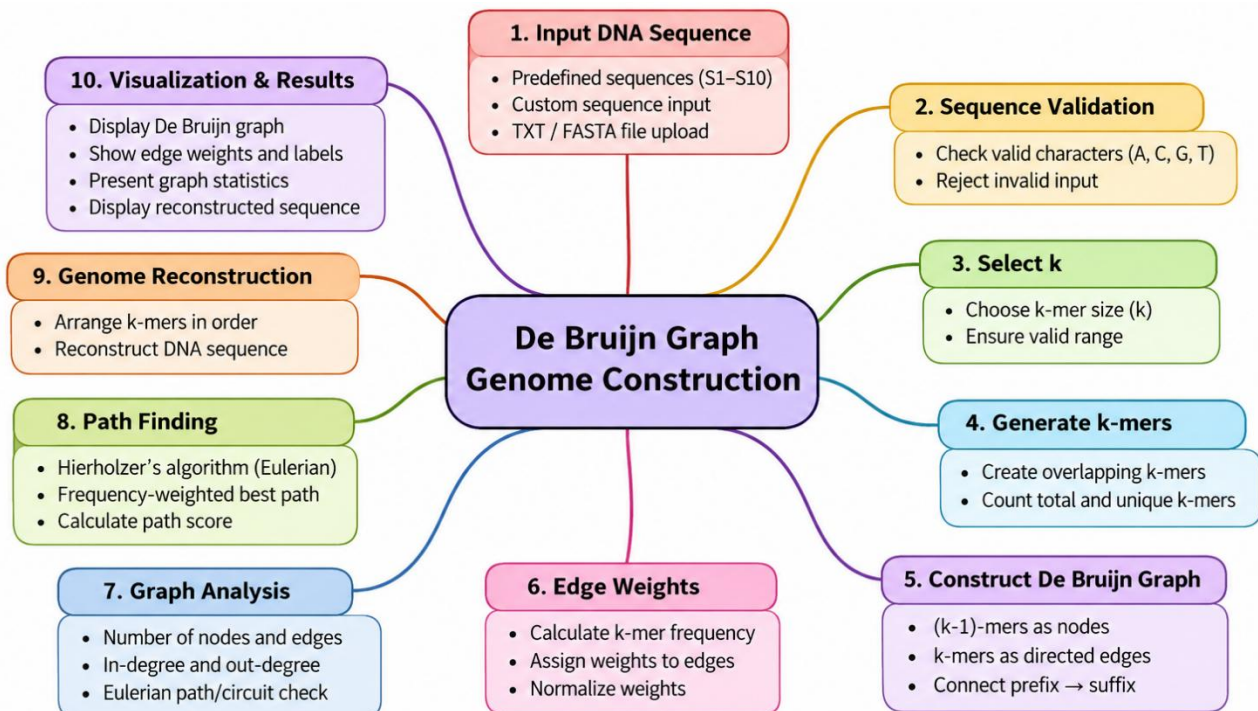
2.1 LIST OF ACTIVITIES

- Study the given DNA sequencing data.
- Select or provide a DNA sequence as input.
- Select a suitable value of k .
- Generate k -mers from the DNA sequence.
- Construct the De Bruijn graph using the k -mers.
- Assign weights to graph edges based on k -mer frequency.
- Analyze the graph and determine a suitable path.
- Reconstruct the DNA sequence from the path.
- Visualize the graph and display the results.

2.2 OBJECTIVES

- To develop a web application for genome construction using a De Bruijn graph.
- To generate k -mers from the given DNA sequence.
- To construct a directed De Bruijn graph from the generated k -mers.
- To calculate edge weights based on the frequency of k -mers.
- To identify a suitable path through the constructed graph.
- To reconstruct the DNA sequence from the obtained path.
- To provide a clear visualization of the graph and its corresponding results.
- To allow users to work with the given sequences as well as custom DNA input.

2.3 MINDMAP



3. MATHEMATICAL MODELING

The genome construction problem is modeled using a **De Bruijn graph**, where the DNA sequence is divided into overlapping k-mers and represented as a directed graph.

3.1 INPUT DATA SEQUENCE

Let the input DNA sequence be:

$$S = s_1 s_2 s_3 \dots s_n$$

where:

- $S = \text{DNA sequence}$
- $n = \text{length of the DNA sequence}$
- $s_i = \{A, C, G, T\}$

For example:

$$S = \text{ATGATGATGCAT}$$

Here,

$$n = 12$$

3.2 SELECTION OF K-MER SIZE

Let k be the selected k-mer size, where:

$$2 \leq k \leq n$$

The number of k-mers generated from the sequence is:

$$m = n - k + 1$$

For $n = 12$ and $k = 3$:

$$m = 12 - 3 + 1 = 10$$

Thus, 10 overlapping k-mers are generated.

3.3 GENERATION OF K-MERS

Each k-mer is represented as:

$$K_i = s_i s_{i+1} \dots s_{i+k-1}$$

For:

$$S = \text{ATGATGATGCAT}, \quad k = 3$$

the k-mers are:

$$\text{ATG, TGA, GAT, ATG, TGA, GAT, ATG, TGC, GCA, CAT}$$

3.4 DE BRUIJN GRAPH CONSTRUCTION

The De Bruijn graph is represented as:

$$G = (V, E)$$

where:

- $V = \text{set of unique } (k - 1)\text{-mers}$
- $E = \text{set of k-mers}$

For each k-mer K :

$$\text{prefix}(K) = \text{first } (k - 1) \text{ characters}$$

$$\text{suffix}(K) = \text{last } (k - 1) \text{ characters}$$

A directed edge is created as:

$$\text{prefix}(K) \rightarrow \text{suffix}(K)$$

For example:

$$K = ATG$$

gives:

$$AT \rightarrow TG$$

Thus, AT and TG are nodes and ATG is the directed edge.

3.5 EDGE WEIGHT CALCULATION

Each graph edge is assigned a weight according to the frequency of its corresponding k-mer in the **current input sequence**.

$$W(e_K) = f(K)$$

where $f(K)$ is the number of occurrences of k-mer K .

where $\setminus(f(K)\setminus)$ is the number of occurrences of k-mer $\setminus(K\setminus)$.

For the example:

<i>k-mer</i>	<i>Weight</i>
ATG	3
TGA	2
GAT	2
TGC	1
GCA	1
CAT	1

For example:

$$W(ATG) = 3$$

3.6 WEIGHT NORMALIZATION

Let the maximum edge weight be:

$$W_{max} = \max_{e \in E} W(e)$$

The normalized edge weight is:

$$W_{norm}(e) = \frac{W(e)}{W_{max}}$$

For the example:

$$W_{max} = 3$$

Therefore:

$$W_{norm}(ATG) = \frac{3}{3} = 1$$

$$W_{norm}(TGA) = \frac{2}{3} = 0.667$$

3.7 GRAPH ANALYSIS

For every node v , the following are calculated:

In-degree:

$$\text{deg}^-(v) = \text{number of incoming edges}$$

Out-degree:

$$\text{deg}^+(v) = \text{number of outgoing edges}$$

These values are used to determine whether a Eulerian path or circuit exists.

3.8 PATH FINDING

If the graph satisfies the Eulerian conditions, **Hierholzer's algorithm** is used to obtain a Eulerian path. The Eulerian path:

$$P = (e_1, e_2, \dots, e_m)$$

uses every edge exactly once.

When a weighted path is required, the score of a path is calculated as:

$$Score(P) = \sum_{e \in P} W(e)$$

A path with a higher total edge weight is preferred.

3.9 GENOME RECONSTRUCTION

Once the k-mers are arranged in the selected path, the DNA sequence is reconstructed.

The first k-mer is taken completely and the last nucleotide of each subsequent k-mer is appended.

For example:

$$ATG \rightarrow TGA \rightarrow GAT \rightarrow ATG$$

produces:

$$ATGATGATG$$

3.10 COMPLETE MATHEMATICAL MODELING

The overall model can be represented as:

$$S, k \rightarrow \{K_1, K_2, \dots, K_m\} \rightarrow G(V, E) \rightarrow G(V, E, W) \rightarrow Path \rightarrow Reconstructed\ DNA$$

where:

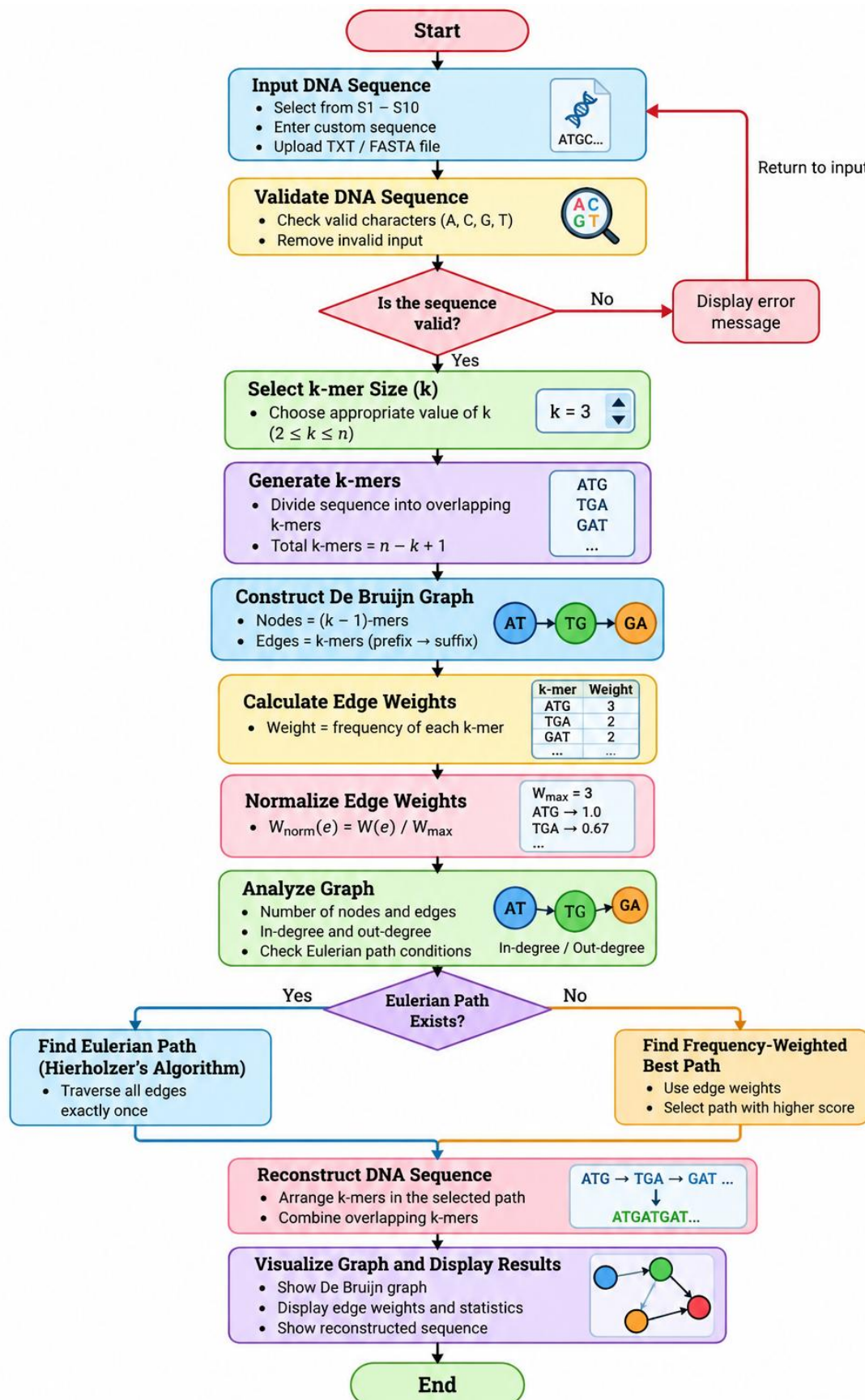
- S = input DNA sequence
- k = selected k-mer size
- K_i = generated k-mers
- $G(V, E)$ = De Bruijn graph
- W = edge weights based on k-mer frequency
- $Path$ = Eulerian or weighted best path
- Reconstructed DNA = final genome sequence obtained from the path.

The mathematical model describes the complete process of genome construction using the De Bruijn graph. It starts with a DNA sequence and a selected k-mer size, from which overlapping k-mers are generated. These k-mers are then used to create the graph, where (k-1)-mers form the nodes and k-mers form the directed edges. Each edge is assigned a weight according to the frequency of its corresponding k-mer in the input sequence. The graph is then analyzed to determine a suitable traversal path using a Eulerian path or a frequency-weighted best path. Finally, the ordered k-mers are combined using their overlapping bases to reconstruct the DNA sequence. Thus, the model provides a systematic mathematical representation of the complete genome construction process from input sequence to final reconstructed sequence.

The model also establishes a clear relationship between the different stages of genome construction. The generated k-mers provide the basic elements for graph formation, while their frequencies determine the edge weights. The graph structure and weights are then used to identify an appropriate path, which represents the possible order of overlapping k-mers. By following this path and combining the overlapping

nucleotides, the final DNA sequence is reconstructed. This mathematical representation helps in understanding and implementing the genome construction process systematically.

4. FLOWCHART



5. IMPLEMENTATION

The De Bruijn Graph Genome Construction system is implemented as a web-based application using Python and FastAPI for the backend and HTML, CSS, JavaScript, and SVG for the frontend. The implementation allows the user to select one of the provided DNA sequences, enter a custom sequence, or upload a TXT/FASTA file.

The implementation follows the mathematical model developed in the previous section. First, the input DNA sequence is validated to ensure that it contains only valid DNA bases (A, C, G, and T). The user then selects the k-mer size, after which overlapping k-mers are generated from the sequence.

The generated k-mers are used to construct the De Bruijn graph. Each unique (k-1)-mer is represented as a node, while every k-mer forms a directed edge between its prefix and suffix nodes. The frequency of each k-mer in the current input sequence is calculated and assigned as the weight of its corresponding edge.

The graph is then analyzed by calculating the number of nodes, edges, and the in-degree and out-degree of each node. The application checks the conditions for a Eulerian path or circuit. When a Eulerian traversal is possible, Hierholzer's algorithm is used to obtain the path. A frequency-weighted best-path approach is also provided for situations where a weighted path is required.

Finally, the selected path is used to reconstruct the DNA sequence by combining the overlapping k-mers. The application displays the reconstructed sequence along with the graph, edge weights, k-mer frequencies, path information, and other calculated statistics.

5.1 DATASET

The application uses the provided DNA sequencing data containing 10 sequences (S1–S10). Each sequence can be selected individually from the dropdown menu. In addition to the predefined sequences, the application allows the user to enter a custom DNA sequence or upload a TXT/FASTA file.

```
>S1
TTGACCGATGACCCCGGTTTCAGGCTTCACCACAGTGTGGAACGCGGTCGTCTCCGAACCTAACGGCGACCCTAAGGT
TGA
CGACGGACCCAGCAGTGATGCTAATCTCAGCGCTCCGCTGACCCCTCAGCAAAGGGCTTGCTCAATCTCGTCCAGC
CAT
TGACCATCGTCGAGGGGTTTGCTCTGTTATCCGTGCCGAGCAGCTTTGTCCAAAACGAAATCGAGCGCCATCTGCGG
GCC
CCGATTACCGACGCTCTCAGCCGCCGACTCGGACATCAGATCCAACCTCGGGGTCCGCATCGCTCCGCCGGCGACCGA
CGA
AGCCGACGACACTACCGTGCCGCCTTCCGAAAATCCTGCTACCACATCGCCAGACACCACAACCGACAACGACGAGA
TTG
ATGACAGCGCTGCGGCACGGGGCGATAACCAGCACAGTTGGCCAAGTTACTTCACCGAGCGCCCGCACAATACCGA
TTCC
GCTACCGCTGGCGTAACCAGCCTTAACCGTCGCTACACCTTTGATACGTTTCGTTATCGGCGCCTCCAACCGGTTGCG
CA
CGCCGCCGCCTTGCGGATCGCAGAAGCACCCGCCGCGCTTACAACCCCTGTTTCATCTGGGGCGAGTCCGGTCTCG
GCA
AGACACACCTGCTACACGCGGCAGGCAACTATGCCAACGGTTGTTCCCGGAATGCGGGTCAAATATGTCTCCACC
GAG
GAATTCACCAACGACTTCATTAACCTCGCTCCGCGATGACCGCAAGGTCGCATTCAAACGCAGCTACCGCGACGTAGA
CGT
```

5.2 CODING

```
    return result
def browser_graph(graph: dict, max_edges: int = 250):
    edges = graph["edges"][:max_edges]
    used_nodes = set()
    for edge in edges:
        used_nodes.add(edge["from"])
        used_nodes.add(edge["to"])
    max_weight = max((e["weight"] for e in graph["edges"]), default=1)
    nodes = [
        {"id": n, "in": graph["indeg"].get(n, 0), "out": graph["outdeg"].get(n, 0)}
        for n in used_nodes
    ]
    browser_edges = [{"*e, "normalized_weight": round(e["weight"] / max_weight, 4)} for e in edges]
    return nodes, browser_edges
def analyze_sequence(sequence: str, sequence_id: str, k: int, source_label: str) -> dict:
    sequence = clean_dna(sequence)
    if not sequence:
        raise ValueError("No valid DNA bases were found. Use only A, C, G and T.")
    if k < 2:
        raise ValueError("k must be at least 2.")
    if len(sequence) < k:
        raise ValueError(f"The input contains {len(sequence)} bases, but k={k}. Choose a smaller k.")
    counts = kmer_counts(sequence, k)
    graph = build_graph(sequence, k, counts)
    euler = eulerian_path(graph)
    best = weighted_best_path(graph)
    nodes, browser_edges = browser_graph(graph)
    max_weight = max((e["weight"] for e in graph["edges"]), default=0)
    all_edges = [
        {"*e, "normalized_weight": round(e["weight"] / max(max_weight, 1), 4)}
        for e in graph["edges"]
    ]
    common = [{"kmer": kmer, "frequency": freq} for kmer, freq in counts.most_common(100)]
    return {
        "sequence_id": sequence_id,
        "source": source_label,
        "sequence": sequence,
        "sequence_length": len(sequence),
        "k": k,
        "nodes_count": len(graph["nodes"]),
        "edges_count": len(graph["edges"]),
        "unique_kmers": len(counts),
        "max_weight": max_weight,
        "eulerian": euler["is_eulerian"],
        "euler_score": euler["score"],
        "euler_edges_used": len(euler["edge_ids"]),
        "euler_reconstruction": reconstruct(graph["edges"], euler["edge_ids"]),
        "best_score": best["score"],
```

```

    "best_edges_used": len(best["edge_ids"]),
    "best_path_nodes": best["nodes"],
    "best_reconstruction": reconstruct(graph["edges"], best["edge_ids"]),
    "best_method": best["method"],
    "nodes": nodes,
    "edges": browser_edges,
    "all_edges": all_edges,
    "common_kmers": common,
}
@app.get("/", response_class=HTMLResponse)
async def home():
    return HTMLResponse((BASE_DIR / "templates" / "index.html").read_text(encoding="utf-8"))
@app.get("/api/sequences")
async def get_sequences():
    return {"sequences": {name: len(seq) for name, seq in SEQUENCES.items()}}
@app.get("/api/sequence-text")
async def get_sequence_text(sequence_id: str = "S1"):
    if sequence_id not in SEQUENCES:
        return JSONResponse({"error": "Sequence not found."}, status_code=404)
    return {"sequence_id": sequence_id, "sequence": SEQUENCES[sequence_id]}
@app.get("/api/analyze")
async def analyze(sequence_id: str = "S1", k: int = 21):
    if sequence_id not in SEQUENCES:
        return JSONResponse({"error": "Sequence not found."}, status_code=404)
    try:
        return analyze_sequence(SEQUENCES[sequence_id], sequence_id, k, f"Dataset read {sequence_id}")
    except ValueError as exc:
        return JSONResponse({"error": str(exc)}, status_code=400)
@app.post("/api/analyze-custom")
async def analyze_custom(payload: dict):
    try:
        sequence = str(payload.get("sequence", ""))
        k = int(payload.get("k", 21))
        return analyze_sequence(sequence, "CUSTOM", k, "Typed DNA sequence")
    except (ValueError, TypeError) as exc:
        return JSONResponse({"error": str(exc)}, status_code=400)
@app.post("/api/upload")
async def upload(file: UploadFile = File(...)):
    global SEQUENCES
    content = await file.read()
    try:
        text = content.decode("utf-8")
    except UnicodeDecodeError:
        text = content.decode("latin-1")
    parsed = parse_sequences(text)
    if not parsed:
        return JSONResponse({"error": "No valid DNA sequences were found in the file."}, status_code=400)
    SEQUENCES = parsed
    DATA_FILE.write_text(text, encoding="utf-8")
    return {"success": True, "sequences": {name: len(seq) for name, seq in SEQUENCES.items()}}

```

```
if __name__ == "__main__":
    import uvicorn
    uvicorn.run("app:app", host="127.0.0.1", port=8000, reload=True)
```

5.3 OUTPUT

a) Taking Inputs of genome sequence and k value:

AI / ML - GENOME ASSEMBLY



Weighted De Bruijn Graph

Genome construction using k-mers, frequency-based edge weights and best-path analysis.

WEEKEND CHALLENGE - 01

⚙️ Dataset & Parameters

Choose one of the 10 reads, enter your own sequence, or upload a TXT / FASTA file.

Sequencing read

S1 • 80 bp

⌵

k-mer size

21

Build Graph →

Selected DNA sequence

S1 - 80 bases

TTGACCGATGACCCCGGTTACAGGCTTCACACAGTGTGGAACGCGGTCGTCTCCGAACCTTAACGGCGACCCCTAAGGTTGA

🔥 Use your own sequence

A • C • G • T

Paste DNA here. Spaces, line breaks and FASTA headers are ignored.

Example: ATGCGTACCGATGCTAGCTAG...

Copy selected sequence

Analyze Typed Sequence

Clear

b) Weighted Path and reconstruction:

INPUT LENGTH

80

base pairs

k-MERS / EDGES

60

total edges

UNIQUE k-MERS

60

distinct patterns

NODES

61

(k-1)-mers

MAX WEIGHT

1

highest frequency

EULERIAN

Yes

all edges usable

🏆 Weighted Best Path

Higher-frequency k-mers contribute more to the path score.

BEST PATH SCORE

60

EDGES IN BEST PATH

60

PATH METHOD

Eulerian traversal with frequency weights

Best Path

TTGACCGATGACCCCGGTTTC → TGACCGATGACCCCGGTTCA → GACCGATGACCCCGGTTTCAG → ACCGATGACCCCGGTTTCAGG → CCGATGACCCCGGTTTCAGGC → CGATGACCCCGGTTTCAGGCT → GATGACCCCGGTTTCAGGCTT → ATGACCCCGGTTTCAGGCTTC → TGACCCCGGTTTCAGGCTTCA → GACCCCGGTTTCAGGCTTCAC → ACCCGGTTTCAGGCTTCACC → CCCCGGTTTCAGGCTTCACCA → CCCGTTTCAGGCTTCACCAC → CCGGTTTCAGGCTTCACCACA → CGGTTTCAGGCTTCACCACAG → GTTTCAGGCTTCACCACAGT → GTTCAGGCTTCACCACAGTG → TTCAGGCTTCACCACAGTGT → TCAGGCTTCACCACAGTGTG → CAGGCTTCACCACAGTGTGG → AGGCTTCACCACAGTGTGGA → GGGCTTCACCACAGTGTGGAA → GCTTCACCACAGTGTGGAAC → CTTTCACCACAGTGTGGAACG → TTCACCACAGTGTGGAACG → TCACCACAGTGTGGAACGCG → CACCACAGTGTGGAACGCGG → ACCACAGTGTGGAACGCGGT → CCACAGTGTGGAACGCGGTC → CACAGTGTGGAACGCGGTCG → ACAGTGTGGAACGCGGTCGT → CAGTGTGGAACGCGGTCGTC → AGTGTGGAACGCGGTCGTC → GTGTGGAACGCGGTCGTCCTC → TGTGGAACGCGGTCGTCCTCC → GTGGAACGCGGTCGTCCTCCG → TGGAACGCGGTCGTCCTCCGA → GGAACGCGGTCGTCCTCCGAA → GAACGCGGTCGTCCTCCGAAC → AACGCGGTCGTCCTCCGAAC → ACGCGGTCGTCCTCCGAACCT → CGCGGTCGTCCTCCGAACCTA → GCGGTCGTCCTCCGAACCTAA → CGGTCGTCCTCCGAACCTAAC → GGTGTCCTCCGAACCTAACG → GTCGTCCTCCGAACCTAACCG → TCGTCTCCGAACCTAACGGC → CGTCTCCGAACCTAACGGCG → GTCTCCGAACCTAACGGCGA → TCTCCGAACCTAACGGCGAC → CTCCGAACCTAACGGCGACC → TCCGAACCTAACGGCGACCC → CCGAACCTAACGGCGACCCCT → CGAACCTAACGGCGACCCCTA → GAACCTAACGGCGACCCCTAA → AACTTAACGGCGACCCCTAAG → ACTTAACGGCGACCCCTAAGG → CTTAACGGCGACCCCTAAGGT → TTAACGGCGACCCCTAAGGTT → TAACGGCGACCCCTAAGGTTG → AACGGCGACCCCTAAGGTTGA

Best Reconstruction

TTGACCGATGACCCCGGTTTCAGGCTTCACACAGTGTGGAACGCGGTCGTCTCCGAACCTAACGGCGACCCCTAAGGTTGA



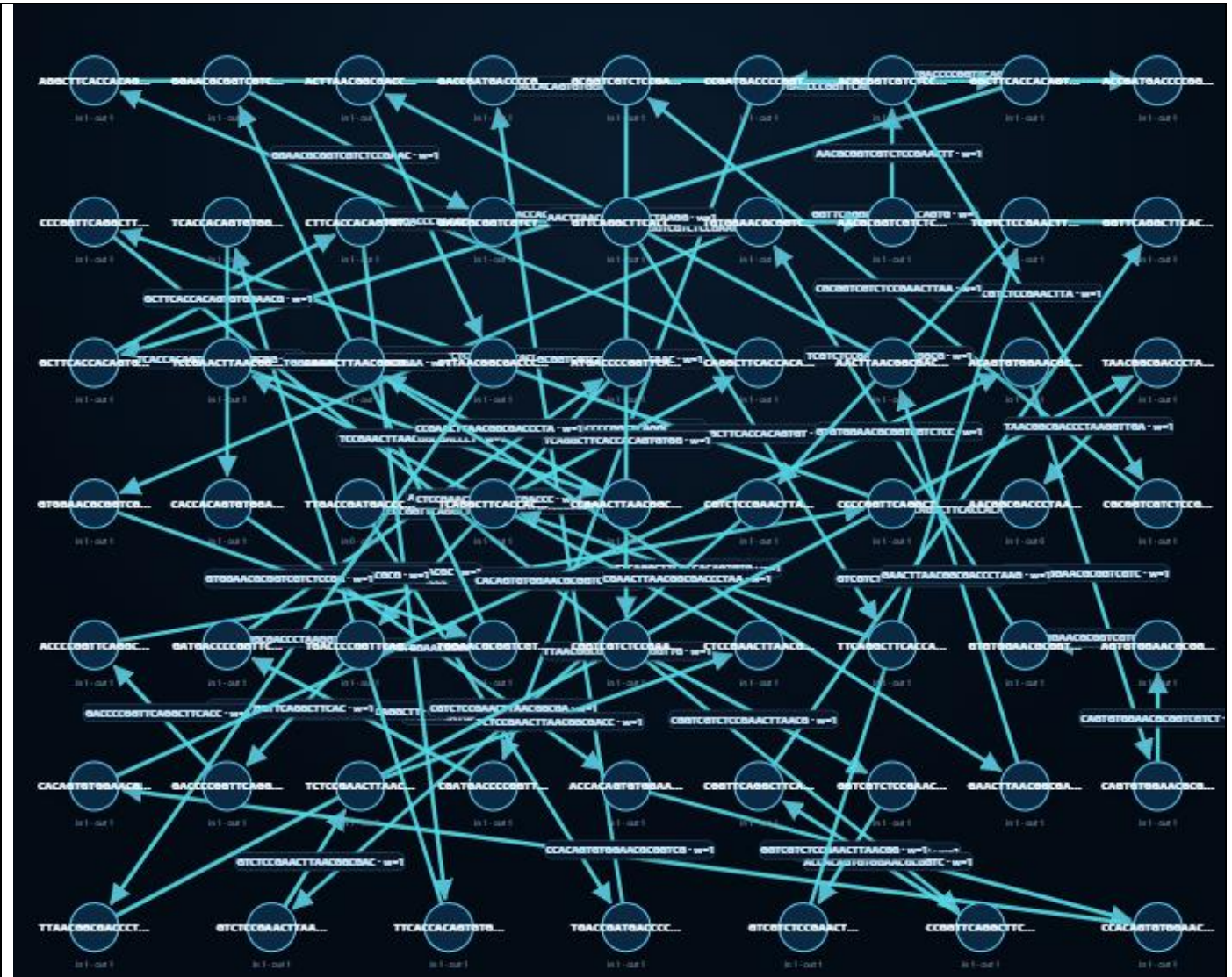
Eulerian Reconstruction

Hierholzer's algorithm attempts to use every graph edge exactly once.

Score **60** Edges Used **60**

TTGACCGATGACCCCGGTTGAGGCTTACCCACAGTGTGGACGCGGTCTCTCCGAACCTAACGGCGACCCCTAAGGTTGA

c) De Bruijn Graph:



d) Weighted Table:



Weighted Edge Table

Weight = frequency of the exact k-mer in the current input only.

ID	k-mer	From	To	Weight	Normalized
0	TTGACCGATGACCCCGGTTCA	TTGACCGATGACCCCGGTTCT	TGACCGATGACCCCGGTTCA	1	1
1	TGACCGATGACCCCGGTTGAG	TGACCGATGACCCCGGTTCA	GACCGATGACCCCGGTTGAG	1	1
2	GACCGATGACCCCGGTTGAGG	GACCGATGACCCCGGTTGAGG	ACCGATGACCCCGGTTGAGG	1	1
3	ACCGATGACCCCGGTTGAGGC	ACCGATGACCCCGGTTGAGG	CGATGACCCCGGTTGAGGC	1	1
4	CGATGACCCCGGTTGAGGCT	CGATGACCCCGGTTGAGGC	GATGACCCCGGTTGAGGCT	1	1
5	GATGACCCCGGTTGAGGCTT	GATGACCCCGGTTGAGGCTT	ATGACCCCGGTTGAGGCTTC	1	1
6	GATGACCCCGGTTGAGGCTTC	GATGACCCCGGTTGAGGCTTC	ATGACCCCGGTTGAGGCTTC	1	1

e) k-mer frequency:

k-mer Frequency Summary Most frequent exact k-mers in the current DNA input.

Rank	k-mer	Frequency
1	TTGACCGATGACCCCGTTCA	1
2	TGACCGATGACCCCGTTCA	1
3	GACCGATGACCCCGTTCA	1
4	ACCGATGACCCCGTTCA	1
5	CCGATGACCCCGTTCA	1
6	CGATGACCCCGTTCA	1
7	GATGACCCCGTTCA	1

6. CONCLUSION

6.1 SUMMARIZATION

The project successfully developed a **Weighted De Bruijn Graph-based Genome Reconstruction System**. The system takes a DNA sequence as input, generates k-mers, calculates their frequencies, and constructs a directed De Bruijn graph using $(k - 1)$ -mers as nodes and k-mers as edges. The k-mer frequency is assigned as the edge weight to represent the occurrence of each k-mer in the input sequence. The system then performs degree analysis and checks whether an Eulerian traversal is possible. For Eulerian cases, the sequence is reconstructed by traversing the graph, while for non-Eulerian cases, a frequency-weighted path search is used to obtain the best possible reconstruction.

The project also provides an interactive **FastAPI-based web application** through which users can select existing sequences, enter custom DNA sequences, upload sequence files, select the value of k , and analyze the generated graph. The application displays k-mer frequencies, graph nodes, edges, weights, path information, and reconstructed sequences. Thus, the project successfully combines graph-based modelling and sequence analysis into a practical genome reconstruction system.

6.2 LIMITATIONS

Despite successfully implementing the proposed approach, the system has certain limitations. The quality of reconstruction depends strongly on the selected value of k and the quality of the input DNA sequence. A very small value of k can produce a highly connected graph with multiple possible paths, whereas a large value of k can reduce the number of overlaps and make reconstruction more difficult.

The current implementation also uses a **bounded weighted search** for non-Eulerian graphs. Therefore, it does not guarantee the globally optimal weighted path for every complex graph. The system is mainly designed for demonstration and analysis of relatively manageable sequences, and processing very large genomic datasets may require additional optimization. Sequencing errors, mutations, repeated regions, and other biological variations can also introduce ambiguity into the reconstruction process.

6.3 FUTURE WORK

The system can be further improved by incorporating more advanced genome assembly techniques and optimization algorithms. Future work can include the implementation of **error correction and noise handling**

to improve reconstruction from real sequencing data. More efficient path-finding and optimization algorithms can also be introduced to handle larger and more complex De Bruijn graphs.

The application can additionally be extended to support **large-scale genomic datasets**, multiple sequencing reads, paired-end reads, and more advanced graph visualization. Automatic selection or recommendation of an appropriate k value can also be implemented based on the characteristics of the input data. In the future, the system can be developed into a more complete genome assembly platform capable of handling real-world sequencing datasets with improved accuracy, scalability, and performance.